picoSargon — Doc + Grumpy en C++ nativo para Raspberry Pi Pico W / Pico 2 W

Documentación técnica del proyecto — base v0.2.10 (19 de octubre de 2026)
Plataformas: Raspberry Pi Pico W (RP2040) y Pico 2 W (RP2350).
Lenguaje: C++17 en firmware y motor; Python 3 para herramientas ejecutadas en Windows; PowerShell para construcción local.
Protocolo: Universal Chess Interface (UCI) sobre TCP/IP; diagnósticos por USB CDC serie.

> **Alcance y estado.** Este documento describe la implementación examinada en `src/`, `include/`, `tools/` y `build.ps1` de picoSargon v0.2.10. La Pico W **sí** ha ejecutado versiones anteriores del motor C++ en partidas reales; las variantes compiladas para Pico 2 W y la integración final de v0.2.10 deben verificarse físicamente. Las mediciones recogidas aquí provienen de telemetrías de v0.2.7 y v0.2.8 y no constituyen benchmarks de v0.2.10. Algunos aspectos del diseño histórico de Doc/Grumpy carecen de documentación de entrenamiento original en este repositorio: se describe su implementación verificable, no una genealogía supuesta.

Índice

1. Motivación, origen y evolución
2. Arquitectura general
3. Modelo del tablero y legalidad
4. Doc: búsqueda, matemáticas y heurísticas
5. Grumpy: red neuronal y evaluación
6. Ciclo UCI, TCP, Wi-Fi, USB y LED
7. Memoria y límites de RP2040/RP2350
8. Configuración centralizada
9. Árbol del proyecto y utilitarios
10. Instalación y compilación en VS Code / Windows
11. Grabación, validación y rendimiento
12. Limitaciones, seguridad y trabajo futuro

---

1. Motivación, origen y evolución

picoSargon es el proyecto autónomo que traslada a microcontroladores Raspberry Pi Pico el jugador de ajedrez **Doc**, con un evaluador neuronal compacto **Grumpy**. Nace de los experimentos de **Sargon-NNUE** y del port posterior **Sargon Micro** escrito en MicroPython, pero **no** reemplaza ni debe modificar esos repositorios.

En la primera adaptación MicroPython se priorizó la compatibilidad con la SRAM y la ejecución interpretada: se simplificaron las optimizaciones del buscador y se restringió la profundidad. Se adoptó un tablero *Mailbox120*. Las pruebas reales evidenciaron una velocidad aproximada de **10–12 nodos/s** y **38,271 segundos para 449 nodos** (2 ply desde inicio). La portabilidad funcional se consiguió, pero el costo del intérprete y de las estructuras dinámicas resultó dominante.

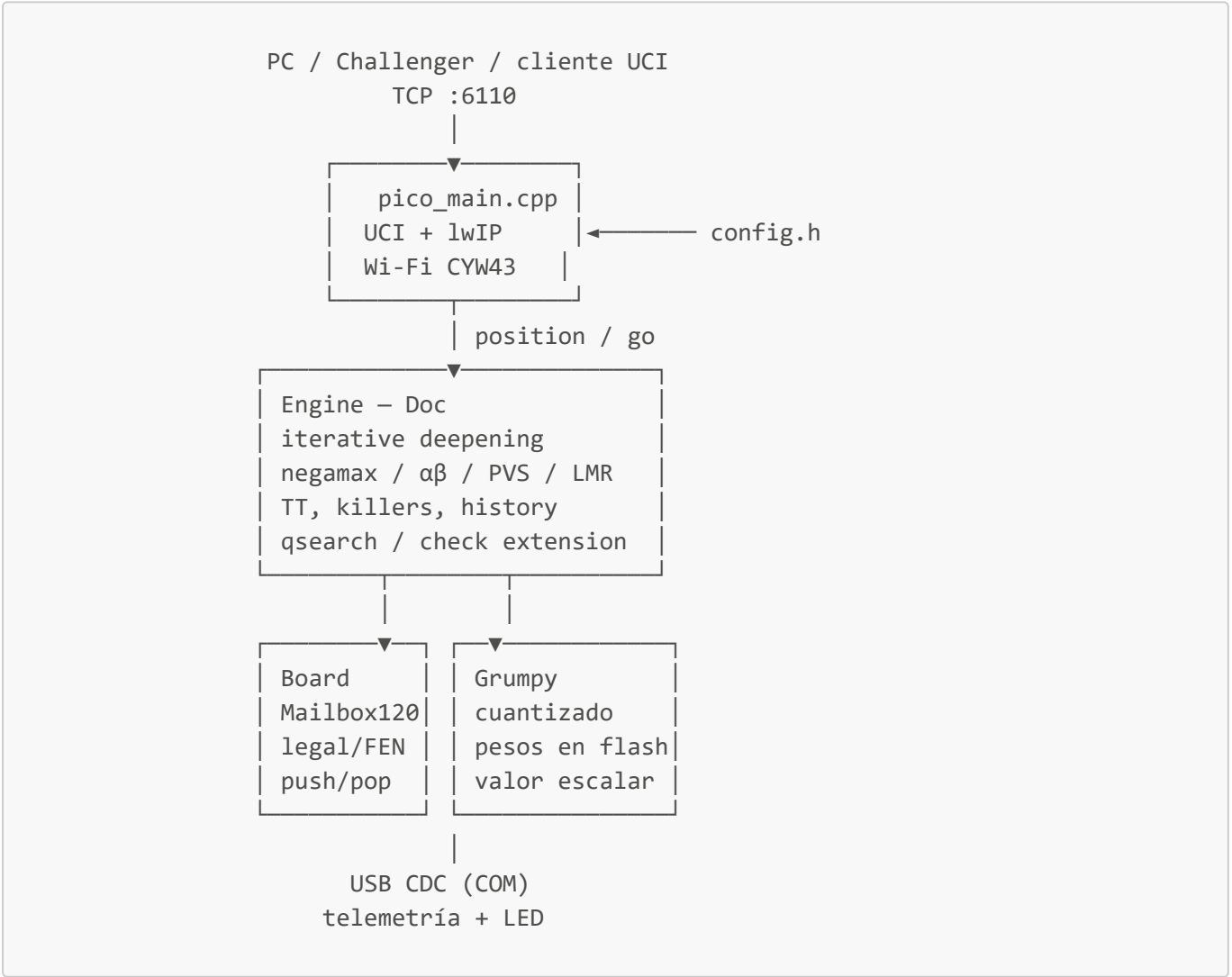
Se creó entonces un núcleo nativo C++17 con equivalencia de reglas, pesos Grumpy y búsqueda a dos niveles como referencia. En una Pico W real, un caso comparable completó **449 nodos en 498 ms (901 NPS)**: mejora temporal de aproximadamente **76,9×** frente a la muestra MicroPython, con la misma jugada inicial `b1c3`. Son **mediciones de ese caso**, no una promesa de aceleración universal. Más adelante se recuperaron PVS, LMR, TT, ordenación y heurísticas, con profundidad principal configurable. La v0.2.10 centraliza parámetros en `config.h`.

En esta línea se diferencia entre:

- **Doc:** política determinística de selección de jugadas mediante búsqueda.
- **Grumpy:** estimador neuronal de la posición evaluada en los nodos.
- **Mailbox120:** reglas de ajedrez, estado y ejecución reversible de movimientos.
- **Pico SDK:** plataforma embebida, USB CDC, Wi-Fi CYW43 y pila TCP/IP lwIP.
- **Herramientas Windows:** compilación con CMake/Ninja y generadores/diagnósticos Python.

No es un motor AlphaZero/MCTS; no genera una distribución aprendida de políticas sobre acciones y **no** utiliza aprendizaje en línea en el microcontrolador.

2. Arquitectura general



Fuentes principales: `src/board.cpp` y `include/board.hpp`; `src/grumpy.cpp` y `include/grumpy.hpp`; `src/engine.cpp` y `include/engine.hpp`; `src/pico_main.cpp`; `src/host_main.cpp` (harness PC). El núcleo

`sargon_core` es compartido entre el ejecutable de PC y el firmware ARM, mientras `pico_main.cpp` encapsula las particularidades del microcontrolador.

La construcción embebida utiliza C++17, Pico SDK (`pico_stdlib`, `pico_cyw43_arch_lwip_poll`) y configura USB CDC habilitado y UART de diagnóstico deshabilitada. En esta arquitectura la inferencia se realiza en CPU; no hay GPU, inferencia delegada ni necesidad de un sistema de archivos de MicroPython.

3. Modelo del tablero y legalidad

3.1 Mailbox120

Se emplea un arreglo de **120 celdas**: 8 filas de juego más un borde centinela. Los índices de las 64 casillas se expresan como `s = 8r + f`, con `r,f ∈ {0,…,7}`. Su índice correspondiente en Mailbox120 es:

$$M(s) = 21 + 10 \left\lfloor \frac{s}{8} \right\rfloor + (s \bmod 8).$$

La celda exterior contiene el centinela `255`, lo que permite comprobar desplazamientos mediante sumas sin numerosas comparaciones de límites. El desplazamiento de un caballo, por ejemplo, se obtiene con $\{-21, -19, -12, -8, 8, 12, 19, 21\}$; el de alfil mediante $\{-11, -9, 9, 11\}$; el de torre mediante $\{-10, -1, 1, 10\}$.

En memoria se utiliza `std::array<uint8_t,120>`. La codificación de piezas dentro del arreglo es `6 + p`, con `p ∈ {-6,…,6}`: 0 representa vacío lógico, números positivos piezas blancas y negativos negras; los tipos 1 a 6 representan peón, caballo, alfil, torre, dama y rey.

3.2 Estado y generación

`Board` conserva el turno `turn`, derechos de enroque, casilla de captura al paso, contador de semijugadas y número de jugada. Acepta FEN y la posición inicial; la interfaz UCI reconstruye el historial completo a partir de `position startpos moves ...` o `position fen ... moves ...`.

La legalidad se establece generando pseudojugadas y descartando las que dejan al rey propio en jaque:

$$\mathcal{L}(s) = \{m \in \mathcal{P}(s) : \neg\,\mathrm{Check}(\mathrm{Apply}(s, m), \mathrm{jugador}(s))\}.$$

`push(m)` registra un `Undo` con estado anterior y hasta cuatro celdas afectadas; `pop()` revierte la jugada. Incluye promociones, enroques, en passant, detección de ataques y finales sin jugadas legales. El arreglo `history_` conserva claves de posición para comprobar triple repetición. `draw()` contempla regla de 50 jugadas, repetición y material insuficiente según la implementación actual.

Costos: aunque el tablero sea compacto, `std::vector<Move>`, `Undo` e historial de claves son estructuras dinámicas. La búsqueda legal puede asignar memoria; la memoria utilizada no es simplemente 120 bytes.

3.3 Pruebas de generación (*perft*)

Perft cuenta hojas del árbol legal sin evaluar posiciones:

$$\mathrm{Perft}(s, d) = \begin{cases} 1 & d = 0, \\ \sum_{m \in \mathcal{L}(s)} \mathrm{Perft}(s_m, d-1) & d > 0. \end{cases}$$

Para la posición inicial los valores de referencia son **20, 400 y 8.902** a profundidades 1, 2 y 3; los tests de PC incluyeron estas comprobaciones. Se requieren suites adicionales de enroque, *en passant*, promoción y repeticiones antes de declarar equivalencia reglamentaria exhaustiva.

4. Doc: búsqueda, matemáticas y heurísticas

4.1 Negamax y alfa-beta

Doc usa búsqueda **negamax**: la calidad de una posición se expresa desde el punto de vista del jugador al turno, de modo que cambiar de turno cambia el signo de la puntuación:

$$V(s, d) = \max_{m \in \mathcal{L}(s)} [-V(s_m, d-1)].$$

Para podar ramas utiliza una ventana $[\alpha, \beta)$. Cada descendiente se consulta con ventana negada $[-\beta, -\alpha]$. Si un valor alcanza o supera β, se puede cortar la exploración de las alternativas restantes de ese nodo. Una puntuación $-30000 + p$ modela mate adverso a una distancia representada por `ply=p`; el código utiliza `MATE=30000` e `INF=1000000`.

Las condiciones terminales tienen prioridad: tablas dan 0 y ausencia de jugadas legales significa mate o ahogado dependiendo de si el rey está en jaque. La búsqueda tiene una profundidad de seguridad `MAX_PLY=40` (independiente de la profundidad principal).

4.2 Profundización iterativa

Se recorren profundidades $1, 2, \ldots, D$, con $D =$ `config::DEPTH`. Cada iteración de raíz guarda la mejor jugada y el valor **solo si toda la iteración ha terminado**. Si llega el timeout o el cliente interrumpe, se devuelve el resultado del último nivel completado; el valor provisional de un nivel incompleto no reemplaza al anterior.

La última mejor jugada se prioriza al empezar la siguiente profundidad. El límite temporal se calcula como $t_{\text{deadline}} = t_0 + T$ cuando `TIMEOUT_MS>0`, y es ilimitado por reloj cuando `TIMEOUT_MS=0`. El motor revisa cancelación/tiempo al menos una vez cada 16 nodos en la búsqueda recursiva y en cada alternativa de raíz.

4.3 Principal Variation Search (PVS)

Después de buscar la primera jugada con ventana completa, las siguientes pueden probarse con una ventana nula aproximada $[\alpha, \alpha + 1)$:

$$v_{\text{scout}} = -V(s_m, d-1, -\alpha - 1, -\alpha).$$

Si $\alpha < v_{\text{scout}} < \beta$, se repite con ventana completa. Esta optimización reduce el trabajo **cuando la ordenación es acertada**, pero en posiciones adversas pueden ser necesarias nuevas búsquedas. La v0.2.10 implementa esa lógica en la raíz y en nodos internos elegibles.

4.4 Late Move Reductions (LMR)

Las jugadas tardías que son tranquilas y no están en nodos críticos pueden buscarse inicialmente con una profundidad reducida. En el código: `depth >= 3`, índice de jugada `>=4`, no captura/promoción, sin jaque propio o resultante, fuera de PV y con suficiente profundidad remanente. Se usa una reducción de **un ply**:

$$V_{\text{reducida}} = -V(s_m, d-2, -\alpha - 1, -\alpha).$$

Si el resultado supera α, se repite a profundidad íntegra. La condición y la fórmula resumen el comportamiento implementado; no se aplica una función adaptativa logarítmica de reducción.

4.5 Quiescencia

La búsqueda de quiescencia evita tomar como definitiva una posición tácticamente inestable al llegar a profundidad principal cero. En posiciones sin jaque se calcula primero la evaluación estática, denominada *stand pat*:

$$q(s, \alpha, \beta, k) = \max\Big(E(s),\ \max_{m \in C(s)} [-q(s_m, -\beta, -\alpha, k-1)]\Big),$$

con $C(s)$ restringida aquí a capturas y promociones. Si $E(s) \geq \beta$, se corta inmediatamente; si $k \leq 0$ se retorna el límite obtenido. La red se evalúa en posición desde el turno correspondiente.

Excepción esencial: cuando el rey está en jaque, `stand pat` no es válido y se deben explorar evasiones legales, incluso si `qdepth` se ha agotado; el código asigna al menos un paso adicional para las evasiones, sujeto al límite `MAX_PLY`. Por ello **`QDEPTH=2` no garantiza que todas las ramas terminen exactamente dos ply después del horizonte principal**.

Al contabilizar nodos: `nodes` es el total visitado por negamax y `qnodes` es el subconjunto con `depth<=0`; no se deben sumar ambos. En las telemetrías v0.2.8 la quiescencia dominó el número de nodos; su fracción no es automáticamente la fracción del tiempo.

4.6 Ordenamiento de jugadas

La prioridad implementada es:

1. Mejor jugada sugerida por la tabla de transposiciones (TT).
2. Capturas y promociones con heurística **MVV-LVA** (*Most Valuable Victim – Least Valuable Attacker*).
3. Dos *killer moves* por ply: jugadas tranquilas que anteriormente provocaron corte.
4. Heurística histórica `history[color][from][to]`, incrementada en cortes de jugadas tranquilas.

Una forma simplificada de la puntuación de capturas es:

$$\text{rank}(m) = 100000 + 10 \cdot \text{valor}(\text{víctima}) - \text{valor}(\text{atacante}) + \text{bonus}_{promo}\,.$$

El ordenamiento se realiza mediante `std::stable_sort`. Los valores clásicos internos utilizados son peón 100, caballo 320, alfil 330, torre 500, dama 900 y rey 20000 para esta heurística, no necesariamente para la evaluación Grumpy.

4.7 Tabla de transposiciones (TT)

La TT tiene **256 entradas** (`TT_SIZE=256`) guardadas en un `std::array`. Cada entrada incluye clave de 64 bits, puntuación, profundidad, tipo de cota y jugada sugerida. La clave usa hash FNV-1a de las 64 celdas normalizadas, el turno, enroque y casilla *en passant*.

$$h_0 = 1469598103934665603, \qquad h_{i+1} = (h_i \oplus b_i) \cdot 1099511628211 \pmod{2^{64}}.$$

Según la profundidad almacenada y la cota, una entrada puede proporcionar valor exacto, cota inferior o superior, o simplemente una jugada para ordenar. Se evita guardar posiciones potencialmente sensibles a

repetición o al contador de 50 jugadas (`halfmove<80` y repetición actual igual a una). Es una política conservadora, no una prueba de inmunidad completa a colisiones o diferencias de historial.

4.8 Otras técnicas

- **Extensión por jaque:** si el nodo principal está en jaque, puede aumentar un ply mientras `ply<12`.
- **Reverse futility pruning:** en nodo no-PV, profundidad 1 y fuera de jaque, un valor estático suficientemente superior a beta (margen 180) permite poda.
- **TT hits/stores/cutoffs:** contadores de diagnóstico global por búsqueda. `cutoffs` agrega varias causas y no aísla su contribución.
- **Sin MCTS ni Null Move:** la implementación examinada no contiene selección PUCT, simulaciones Monte Carlo ni poda por jugada nula.

5. Grumpy: red neuronal y evaluación

5.1 Origen y formato

Grumpy es el **evaluador neuronal compacto preexistente** incorporado al port de Doc; **picoSargon lo consume, no lo entrena**. La referencia disponible es `reference/grumpy.py` y los pesos originales `assets/grumpy.bin`. No se encuentran en este proyecto el dataset, procedimiento de entrenamiento ni métricas de validación originales, por lo que no se atribuyen una arquitectura de entrenamiento o calidad estadística no verificadas.

`grumpy.bin` mide **1.304 bytes** y debe empezar por `GRMP0001`. Contiene **44 bytes de cabecera** seguidos de **1.260 bytes de carga útil**. El decodificador transforma bloques de 12 bytes (96 bits) en 16 enteros de 6 bits, obteniendo:

$$\frac{1260}{12} \cdot 16 = 1680 \text{ coeficientes}, \qquad w_i \in \{0, \dots, 63\}.$$

La cuantización se centra en torno a 31; el valor firmado usado por el evaluador suele ser $\widetilde{w}_i = w_i - 31$, sin descompresión a floats. La tabla decodificada se guarda en `std::array<uint8_t,1680>`.

5.2 Arquitectura **realmente inferida del código**

La implementación realiza una evaluación incremental por características de pieza y casilla, **dos acumuladores de 8 componentes**, activación limitada tipo ReLU y una combinación lineal de 16 componentes dependiente de un **bucket de cantidad de piezas**.

En términos estructurales:

```
Tablero + lado a mover
        |
  pieza / color / casilla
        |
  dos acumuladores [8] + [8]
        |
  selección de perspectiva del turno
        |
  clamp(x, 0, 32) × 16
        |
```

```
8 cabezales de salida (bucket)
      |
escalar neural + material
      |
puntuación entera en centipeones aprox.
```

Es **un evaluador NNUE-like pequeño**, no una red LC0 convolucional, ni una política P/V, ni una red con WDL explícito. Tampoco debe describirse sin reservas como una matriz densa estándar de entrada 768 × 8, porque el indexado de pesos **factoriza/suma componentes por fila y columna**:

$$\widetilde{W}_{f,j} = \widetilde{w}_{a(f,j)} + \widetilde{w}_{b(f,j)}.$$

La función `src/grumpy.cpp` establece los índices exactos `a` y `b`; una representación densa genérica perdería esa estructura. El índice de característica considera color, tipo (seis tipos) y una de 64 casillas; para cada pieza del tablero se actualizan ambos acumuladores con transformaciones espaciales dependientes del color y de la perspectiva.

5.3 Acumuladores y activación

Para cada componente $j \in \{0, \dots, 7\}$, los acumuladores parten de un sesgo de pesos decodificados:

$$A_j = b_j + \sum_{p \in s} W^{(A)}_{f(p),j}, \qquad B_j = b_j + \sum_{p \in s} W^{(B)}_{f'(p),j}.$$

El código utiliza efectivamente índices `raw_[1536+j]` para sesgos y cálculo de índices comprimidos para las contribuciones. En cada llamada se **reconstruyen los acumuladores recorriendo las piezas**; no se mantienen acumuladores actualizados incrementalmente durante `push/pop`.

Sea n el número de piezas presentes. El bucket de salida es:

$$k = \left\lfloor \frac{n - 2}{4} \right\rfloor, \qquad k \in \{0, \dots, 7\}.$$

Por tanto, se utilizan ocho configuraciones de salida según el material remanente. El código rechaza buckets fuera de ese rango. Con $\phi(x) = \min(32, \max(0, x))$, los 16 componentes se ordenan desde la perspectiva del lado al turno:

$$z = (A_0, \dots, A_7, B_0, \dots, B_7) \quad \text{o} \quad z = (B_0, \dots, B_7, A_0, \dots, A_7).$$

La salida neuronal en unidades internas es:

$$T_k = 8 b_k + \sum_{i=0}^{15} \phi(z_i)\, v_{k,i},$$

con $b_k = \texttt{raw[1672+k]} - 31$ y $v_{k,i} = \texttt{raw[1544+16*k+i]} - 31$.

5.4 Escala y combinación con material

La transformación entera de escala que aplica el código es, salvo redondeo entero hacia cero:

$$N(s) = \operatorname{sgn}(T_k) \left\lfloor \frac{|400 T_k|}{1024} \right\rfloor.$$

A ese valor se suma un término de material codificado mediante la constante compacta `0x3847D12C4B064ULL` que extrae grupos de 10 bits según tipo de pieza, con la convención de signo del bucle actual. El resultado de `Grumpy::evaluate(Board)` se retorna finalmente con orientación **blancas positivas / negras negativas**. El buscador lo transforma a perspectiva del turno multiplicando por `b.turn`:

$$E_{\text{turno}}(s) = \texttt{Grumpy::evaluate}(s) \cdot \texttt{turn}(s).$$

Precaución conceptual: esta es la traducción del código real, no una definición abstracta equivalente al diseño original del autor. Conviene mantener una batería de evaluaciones de referencia antes de cambiar codificación, orientación, material, sesgos o buckets. La posición inicial produjo **15** en la prueba de equivalencia inicial.

5.5 Peso en flash y generación

`tools/embed_grumpy.py` lee el binario, comprueba longitud y firma y genera `include/generated_grumpy.hpp`, con `GRUMPY_DATA[]` y `GRUMPY_LENGTH`. El firmware llama a `neural.load_blob(...)` durante el arranque. La tabla empacada reside en el programa; el objeto `Grumpy` aloja la versión desempaquetada en RAM. El proceso no necesita SD, MicroPython ni escritura del archivo de pesos en la placa.

Para regenerar el encabezado en Windows:

```
& 'C:\Users\pedro\ufasta\Scripts\python.exe' .\tools\embed_grumpy.py
```

`build.ps1` ya realiza esta operación automáticamente.

6. Ciclo UCI, TCP, Wi-Fi, USB y LED

6.1 Inicio de firmware

1. `stdio_init_all()` activa USB CDC.
2. **Comportamiento provisional v0.2.10:** se bloquea en `while (!stdio_usb_connected())` hasta que Windows abre el monitor COM; garantiza que no se pierda el mensaje inicial ni la IP. Sin terminal abierto **no habrá Wi-Fi ni UCI**.
3. Inicializa CYW43, LED y pesos Grumpy.
4. Conecta a Wi-Fi mediante SSID y contraseña de `config.h` (WPA2 AES PSK).
5. Crea servidor TCP en `PORT` y publica IP DHCP por USB.
6. Atiende UCI y ejecuta las búsquedas pendientes desde el bucle principal.

El servicio de red utiliza **modo polling**: `cyw43_arch_poll()`. La búsqueda llama regularmente al servicio, lo que permite atender `stop`, `quit` y la conexión mientras calcula, sin realizar la búsqueda dentro del callback de recepción TCP.

6.2 Comandos y semántica

- `uci` → `id name`, `id author`, `uciok`.
- `isready` → `readyok`.
- `ucinewgame` → reinicia el tablero lógico.

- `position startpos [moves …]`, `position fen <FEN> [moves …]` → recrean el tablero y aplican secuencialmente jugadas legales; si falla la reconstrucción, `go` queda bloqueado hasta recibir posición válida.
- `go [wtime … btime …]` → dispara búsqueda con **`TIMEOUT_MS` fijo de `config.h`** y `DEPTH`; en esta base los tiempos incluidos en el mensaje UCI no ajustan dinámicamente ese límite.
- `stop` → solicita interrupción, y se reporta el último nivel completado.
- `quit` → solicita salida/desconexión de sesión.

Limitación: el parser es una implementación UCI **parcial**; no expone toda la negociación de `setoption` ni todos los parámetros de `go`. La identificación `UCI_NAME` es editable. Los mensajes de telemetría se envían por **USB**, no se mezclan indiscriminadamente con el canal de protocolo TCP.

6.3 Telemetría

```
picoSargon C++ v0.2.10 BOOT (USB debug connected)
WiFi connecting SSID=...
UCI ready TCP port 6110 IP 192.168.x.y
SEARCH begin max_depth=4 qdepth=2 timeout_ms=15000
LEVEL depth=1 status=complete elapsed_ms=... nodes=... qnodes=... heap_used=...
heap_free=... stack_gap_approx=...
LEVEL depth=2 status=complete elapsed_ms=... nodes=... qnodes=... heap_used=...
heap_free=... stack_gap_approx=...
SEARCH elapsed_ms=... nodes=... qnodes=... nps=... depth=... tt_hits=...
tt_stores=... cutoffs=... reason=...
UCI >> bestmove ...
```

`elapsed_ms` de cada `LEVEL` corresponde a **ese nivel, no al acumulado**. `nodes` y `qnodes` de `LEVEL` también son por iteración. En `SEARCH` son totales acumulados. `nps` es `nodes × 1000 / elapsed_ms` (valor truncado). `reason=depth-limit` significa profundidad objetivo completada; `reason=stop` significa búsqueda interrumpida por tiempo o cancelación, según el origen observable.

6.4 LED

La implementación utiliza el LED integrado controlado por `cyw43_arch_gpio_put`:

Estado	Señal	Significado
`Init`	Encendido fijo	Inicialización
`Ready`	Conmutación cada 1.000 ms	TCP disponible
`Thinking`	Conmutación cada 3.000 ms	Calculando jugada
`Fatal`	Conmutación cada 100 ms	Error fatal de Wi-Fi o pesos, cuando el controlador está disponible

El intervalo es entre **cambios de estado** del LED, no necesariamente la duración completa de un ciclo encendido/apagado.

7. Memoria y límites de RP2040/RP2350

7.1 Pico W original (RP2040)

- CPU: doble núcleo Arm Cortex-M0+ hasta ~133 MHz nominales; el motor actual ejecuta la búsqueda principalmente en **un núcleo**.
- SRAM total integrada: **264 KiB**, repartida físicamente en bancos; no toda queda libre para C++.
- Flash típica de Pico W: **2 MiB** externa QSPI para firmware, tablas y pesos embebidos.
- Wi-Fi 2,4 GHz mediante CYW43439; LED integrado gestionado por CYW43.
- No hay acelerador neuronal especializado.

La TT de **256 entradas** y los arrays de ordenamiento tienen un tamaño deliberadamente moderado. También existen buffers TCP/Wi-Fi, pilas, runtime C++, historial y vectores, por lo que la SRAM consumida por el proceso es superior al `heap_used` expuesto en telemetría.

7.2 Pico 2 W (RP2350)

- Microcontrolador RP2350, con núcleos seleccionables Arm Cortex-M33 o Hazard3 según firmware/plataforma; el objetivo habitual del Pico SDK C++ aquí es **Arm Cortex-M33**.
- SRAM integrada: **520 KiB**.
- Flash de placa típica: **4 MiB**.
- Mayor capacidad de cómputo y memoria que Pico W; el ecosistema `pico_cyw43_arch_lwip_poll` también permite Wi-Fi.
- Hay que compilar específicamente con `PICO_BOARD=pico2_w`; **el `.uf2` de Pico W no es intercambiable**.

Las cifras de hardware provienen de las especificaciones publicadas de las placas y no describen necesariamente memoria disponible para el ejecutable. La compilación Pico 2 W requiere toolchain compatible con RP2350; GNU Arm 11.2 se usó para RP2040, pero la validación cruzada de Pico 2 W es una tarea pendiente.

7.3 Qué miden los contadores de memoria

En `pico_main.cpp`, `mallinfo()` devuelve:

- `heap_used = uordblks`: bytes de bloques actualmente ocupados en el allocator.
- `heap_free = fordblks`: bytes libres **dentro del heap administrado**; no equivale a SRAM total disponible.
- `stack_gap_approx`: diferencia aproximada entre un marcador local en la pila y `sbrk(0)`.

La medición `stack_gap_approx` depende del layout de memoria y del instante de observación. **No es un watermark preciso de pila ni una garantía de espacio utilizable**. Un aumento de `heap_free` puede reflejar crecimiento de la zona reservada para el heap, no una liberación milagrosa de SRAM; para medir consumo máximo real hacen falta high-watermarks de pila, control de heap y pruebas de estrés.

La telemetría de búsquedas reales v0.2.8 mostró un máximo observado de heap utilizado del orden de **16,6 KiB** y una separación mínima estimada de aproximadamente **105 KiB** durante una partida, sin agotamiento reportado. Son observaciones de esa ejecución; una búsqueda de 4 ply con otra posición o una configuración más agresiva puede tener un máximo diferente.

8. Configuración centralizada

La única fuente de configuración de firmware en v0.2.10 es `include/config.h`. Editarlo desde VS Code y recompilar; no hace falta ejecutar el antiguo generador para definir Wi-Fi o plies.

```
#pragma once
namespace picoSargon { namespace config {
inline constexpr const char* UCI_NAME = "picoSargon v0.2.10 C++";
inline constexpr const char* SSID = "MI_RED_WIFI";
inline constexpr const char* PASSWORD = "MI_PASSWORD_EN_TEXTO_CLARO";
inline constexpr unsigned PORT = 6110;
inline constexpr unsigned TIMEOUT_MS = 15000; // 0 = sin limite de tiempo
inline constexpr int DEPTH = 4;
inline constexpr int QDEPTH = 2;
inline constexpr bool DEBUG = false;          // reservado para uso futuro
} }
```

Elemento	Destino	Observación
`UCI_NAME`	`id name` UCI	Nombre del motor, no cambia nombre de archivo `.uf2`
`SSID`, `PASSWORD`	Conexión CYW43	Contraseña se incorpora **en claro al binario**
`PORT`	Servidor TCP	Ej. 6110
`TIMEOUT_MS`	Tiempo fijo por búsqueda	0 desactiva el reloj; persisten `stop` y `quit`
`DEPTH`	Iteración máxima Doc	Plies principales; valor recomendado 4
`QDEPTH`	Extensión táctica	Valor recomendado 2; jaques pueden extender más
`DEBUG`	Reservado	La bandera existe pero **no habilita funcionalidad adicional todavía**

> **Seguridad:** `include/config.h` está excluido de Git por `.gitignore` al contener una contraseña en texto claro. Mantenerlo respaldado localmente antes de sustituir carpetas o paquetes; nunca adjuntarlo a reportes públicos. El archivo de ejemplo no debe contener claves verdaderas. El servidor UCI TCP no implementa autenticación ni cifrado; úsese en una LAN confiable, no expuesto a Internet.

El script antiguo `tools/generate_config.py` está **obsoleto intencionalmente** y sale con código de error 1 indicando que se edite `config.h`. `tools/embed_grumpy.py` **sí** continúa activo.

9. Árbol del proyecto y utilitarios

```
picoSargon/
├── .vscode/
│   ├── extensions.json           Extensiones sugeridas
│   ├── settings.json             Ajustes del editor
```

```
│   └── tasks.json                Build Pico W / Pico 2 W desde VS Code
├── assets/
│   └── grumpy.bin                Pesos originales, 1304 bytes
├── config/
│   └── lwipopts.h                Parámetros de lwIP
├── include/
│   ├── board.hpp                 Interfaz de tablero
│   ├── engine.hpp                Búsqueda / resultados / memoria
│   ├── grumpy.hpp                Evaluador neuronal
│   ├── config.h                  CONFIGURACIÓN EDITABLE; PRIVADA
│   └── generated_grumpy.hpp      Salida de embed_grumpy.py
├── reference/                    Referencia MicroPython histórica
├── src/
│   ├── board.cpp
│   ├── engine.cpp
│   ├── grumpy.cpp
│   ├── host_main.cpp
│   └── pico_main.cpp
├── tests/
│   └── test_host.py              Prueba PC: perft y UCI
├── tools/
│   ├── embed_grumpy.py           Binario Grumpy -> include C++
│   ├── tcp_uci_test.py           Cliente UCI TCP de prueba
│   └── generate_config.py        Obsoleto en v0.2.10
├── build.ps1                     Compilación Windows
├── CMakeLists.txt                Targets PC y firmware
├── pico_sdk_import.cmake        Integración Pico SDK
└── README.md                     Este documento
```

Herramientas Windows:

- `build.ps1 [-Board pico_w|pico2_w]`: busca SDK local, referencias absolutas a CMake/Ninja/compilador ARM, encuentra `pioasm`/`picotool`, incrusta Grumpy, configura y ejecuta Ninja, y verifica la existencia del UF2. Las variables de entorno establecidas por el script son de **su proceso**, no de la configuración global del sistema.
- `embed_grumpy.py`: recibe `--input` y `--output` opcionales; valida pesos y escribe un arreglo constante en `include/generated_grumpy.hpp`.
- `tcp_uci_test.py <IP> --port 6110 --timeout 180`: abre TCP, negocia `uci`/`isready`, envía `position startpos`, ejecuta `go`, espera `bestmove`, informa tiempo total y cierra.
- `tests/test_host.py [ruta_ejecutable]`: ejecuta perft 1–3 y un intercambio UCI con el ejecutable host. Requiere Python en Windows y un compilador **nativo de PC** si se va a construir ese ejecutable.

No se requiere PowerShell en la placa; no se requiere intérprete Python en el firmware. Visual Studio Code es el editor/orquestador; CMake y Ninja realizan la construcción efectiva.

10. Instalación y compilación en VS Code / Windows

10.1 Requisitos verificados en el entorno de trabajo

Herramienta	Ruta local usada en `build.ps1`
Windows 11 + VS Code	Editor y terminal PowerShell
CMake	`C:\Program Files\CMake\bin\cmake.exe`
Ninja	`C:\Users\pedro\ufasta\Scripts\ninja.exe`
Python	`C:\Users\pedro\ufasta\Scripts\python.exe`
GNU Arm Embedded	`C:\Program Files (x86)\Arm GNU Toolchain arm-none-eabi\11.2 2022.02\bin`
Pico SDK	`C:\Users\pedro\.pico-sdk\sdk\2.3.1`
`pioasm`	`C:\Users\pedro\.pico-sdk\tools\2.3.1\pioasm`
`picotool`	`C:\Users\pedro\.pico-sdk\picotool\2.3.1\picotool`

Estas son **rutas del entorno utilizado**, no ubicaciones universales. Al instalar en otra PC hay que editar `build.ps1`. Para RP2350 podría ser necesario actualizar la cadena de herramientas ARM.

10.2 Apertura del proyecto

1. Extraer el proyecto en una carpeta de trabajo, por ejemplo `C:\Users\pedro\github\Bobby\Akkad\picoSargon`.
2. Abrir **VS Code** → **File** → **Open Folder**, seleccionando exactamente la carpeta que contiene `CMakeLists.txt`.
3. Conservar la estructura `src`, `include`, `config`, `tools` y `.vscode`.
4. Tener habilitadas las extensiones **Raspberry Pi Pico**, **CMake Tools**, **C/C++** y **Python** (la extensión Pico es útil, pero no obligatoria para el script explícito).
5. Abrir **Terminal** → **New Terminal** (PowerShell). **No crear un proyecto Pico nuevo mediante un asistente**, porque este ya tiene un `CMakeLists.txt` propio.

10.3 Configurar el firmware

Editar `include/config.h`: nombre UCI, SSID, PASSWORD, PORT, `TIMEOUT_MS`, `DEPTH=4`, `QDEPTH=2`, `DEBUG=false`. Guardar el archivo; **no ejecutar `tools/generate_config.py`**. Mantener contraseña fuera de repositorios públicos.

10.4 Construir para **Pico W (RP2040)**

En la terminal integrada:

```
cd C:\Users\pedro\github\Bobby\Akkad\picoSargon
.\build.ps1 -Board pico_w
```

El script invoca en orden: validaciones → `embed_grumpy.py` → `cmake -S … -B … -G Ninja -DPICOSARGON_PICO=ON -DPICO_BOARD=pico_w …` → `cmake --build … --parallel 4` → comprobación de UF2.

La versión examinada utiliza el directorio `build-pico-w-v025`, heredado de una etapa previa del script. La etiqueta del directorio **no representa la versión actual del firmware**. Archivo esperado:

```
build-pico-w-v025\picoSargon.uf2
```

Para reconocer la placa correcta, la salida CMake debe mostrar `PICO_BOARD=pico_w` y plataforma `rp2040`. En las compilaciones sucesivas Ninja reutiliza objetos válidos, recompilando solamente los modificados.

10.5 Construir para **Pico 2 W (RP2350)**

```
.\build.ps1 -Board pico2_w
```

El directorio esperado es `build-pico2-w-v025`, pero **debe verificarse toolchain y soporte efectivo de RP2350**. No debe flashearse la imagen de otra placa. La validación de este target en Windows y hardware queda pendiente.

10.6 Compilar mediante tareas del editor

`Terminal → Run Task…` contiene las tareas:

- `picoSargon: Build Pico W (build.ps1)` (por defecto con **Ctrl+Shift+B**).
- `picoSargon: Build Pico 2 W (build.ps1)`.

Estas tareas llaman `powershell.exe -NoProfile -ExecutionPolicy Bypass -File build.ps1 -Board …` con directorio de trabajo raíz. **No es necesario importar el proyecto desde la extensión Pico** ni modificar el `PATH` permanente de Windows. Si el importador de la extensión no encuentra CMake, la compilación con `build.ps1` puede seguir funcionando porque usa la ruta absoluta al ejecutable.

10.7 Si falla la compilación

Revisar **el primer `FAILED` o `error:` real**, no solamente la última excepción `Fallo la compilacion` de PowerShell. Casos observados y resueltos en la evolución:

Mensaje	Interpretación
`lwipopts.h: No such file`	Falta `config/lwipopts.h` o su include en CMake
`No CMAKE_CXX_COMPILER` en pioasm/picotool	El SDK intenta compilar herramientas para **Windows**, distintas del compilador ARM; usar ejecutables y paquetes CMake precompilados
`pioasmConfig.cmake` no encontrado	Nombre **CamelCase**; no confundir con `pioasm-config.cmake`
`'runtime_error' is not a member of std`	Falta `<stdexcept>` en fuente C++
Nota GCC sobre `vector.tcc` y cambio ABI desde 7.1	**No es error**; `-Wno-psabi` silencia esa nota sin eliminar `-Wall`/`-Wextra`

Mensaje	Interpretación
`PICO_BOARD=pico`	Seleccionada placa sin Wi-Fi; para Pico W debe ser `pico_w`

No se debe borrar ni sustituir el SDK instalado para corregir defectos del código del proyecto. Solo regenerar una caché de build si cambió la arquitectura o cadena de compilación y CMake no puede reconducirla.

11. Grabación, validación y rendimiento

11.1 Instalar `.uf2`

1. Desconectar Pico W de USB.
2. Mantener **BOOTSEL** presionado y conectar USB.
3. Soltar al aparecer la unidad `RPI-RP2`.
4. Copiar `build-pico-w-v025\picoSargon.uf2` a `RPI-RP2`.
5. Esperar que la placa reinicie y abrir el monitor serie (puerto COM USB CDC de Windows).

La transferencia UF2 sustituye el programa ejecutado, no es una subida MicroPython por archivos. Conservar el firmware antiguo como respaldo y no subir el proyecto entero mediante herramientas MicroPython.

11.2 Primer arranque

Como la v0.2.10 espera **que se abra el terminal COM**, hay que abrirlo después de flashear: hasta entonces el firmware permanece detenido deliberadamente. La velocidad seleccionada del monitor puede ser 115200 baudios como convención de USB CDC. Al abrirlo deben aparecer `BOOT`, `WiFi connecting …` y `UCI ready TCP port … IP …`.

No interpretar `position startpos moves …` repetido por Challenger como reinicio: el protocolo UCI normalmente reconstruye el tablero desde el inicio y reaplica todo el historial. `ucinewgame` o un nuevo inicio de conexión sí representan eventos de sesión separados.

11.3 Verificación de red y protocolo

Desde otra consola de Windows:

```
Test-NetConnection 192.168.68.105 -Port 6110
& 'C:\Users\pedro\ufasta\Scripts\python.exe' .\tools\tcp_uci_test.py
192.168.68.105 --port 6110 --timeout 180
```

Sustituir IP por la publicada en el puerto serie. El programa debe devolver `uciok`, `readyok`, `info depth …` y `bestmove …`. Verificar que la jugada pertenece al conjunto legal. El valor `--timeout 180` es **del cliente Windows**, distinto de `TIMEOUT_MS` del firmware.

11.4 Pruebas en PC

La compilación host usa el mismo núcleo C++ pero una interfaz UCI de pruebas. En un entorno con compilador nativo adecuado:

```
& 'C:\Program Files\CMake\bin\cmake.exe' -S . -B build-host -G Ninja -
DPICOSARGON_PICO=OFF
& 'C:\Program Files\CMake\bin\cmake.exe' --build build-host
& 'C:\Users\pedro\ufasta\Scripts\python.exe' tests\test_host.py build-
host\picoSargon_host.exe
```

Si no se dispone de compilador C++ nativo para Windows, esos comandos no funcionan solo con ARM GNU: primero hay que proporcionar MSVC/MinGW/Clang host. La prueba `tests/test_host.py` espera perft inicial 20/400/8902 y negociación básica UCI.

11.5 Medidas reales de referencia

Caso	Configuración	Resultado observado
Sargon Micro (MicroPython, Pico W)	2 ply, base anterior	449 nodos, 38.271 ms, ~11,7 NPS
picoSargon C++ v0.2.7 (Pico W)	2 ply, sin heurísticas posteriores	449 nodos, **498 ms**, 901 NPS; `b1c3`
picoSargon v0.2.8, partida de 2 ply	Optimizado, profundidad efectiva 2	Decenas de respuestas subsegundo; velocidad variable en torno a centenares de NPS
picoSargon v0.2.8, partida con 4 ply	`max_depth=4`, timeout 15 s	Varias búsquedas llegaron a 4 ply; muchas se interrumpieron a 3, conservando última iteración completa; resultado observado: mate en Challenger

No extrapolar los tiempos de 2 a 4 ply con un factor constante: crecimiento del árbol, poda alfa-beta, ordenamiento, quiescencia, repetición y configuración de la red alteran el número de nodos. Los contadores de `qnodes` no miden directamente duración de la red, y la telemetría actual no separa microtiempos de evaluación/grabación de jugadas.

12. Limitaciones, seguridad y trabajo futuro

12.1 Limitaciones presentes

- **USB imprescindible para arrancar** durante la fase de diagnóstico por la espera a `stdio_usb_connected()`; futuro: timeout de arranque o reimpresión de IP a demanda.
- Configuración **estática compilada** en `config.h`; cambios de SSID/plies requieren reconstruir firmware y flashear.
- Contraseña Wi-Fi **en texto claro** en fuente y firmware; no usar este diseño como gestor de secretos.
- UCI **parcial**; `go` utiliza presupuesto fijo y no administra automáticamente reloj restante, incremento, `movetime` ni opciones negociadas.
- Telemetría `stack_gap_approx` estimada, **no** máximo real de pila ni memoria libre total.
- Tabla de transposiciones compacta y con hash de 64 bits; no representa almacenamiento perfecto ni elimina toda incertidumbre de colisiones.

- Vectores dinámicos y reconstrucción del historial pueden crecer con partidas prolongadas; las partidas probadas no prueban ausencia de agotamiento futuro.
- `qdepth=2` se limita a la quiescencia de capturas/promociones cuando no hay jaque; las evasiones pueden extenderse.
- No hay soporte de depuración multicore ni validación física certificada para Pico 2 W en esta base.

12.2 Investigaciones recomendadas

1. **Perfilado:** separar `eval_ms`, `legal_ms`, `push/pop_ms`, `qsearch_ms` y costos de pila TCP.
2. **Ensayos equivalentes:** mismas FEN, mismos pesos, `DEPTH=4`, `QDEPTH=2`, mismo timeout, para comparar nodos/jugadas antes y después de cada cambio.
3. **Memoria:** medir high watermark de pila y heap y estrés con historiales largos; considerar reservar vectores, buffers y estructuras para reducir fragmentación.
4. **Quiescencia:** generación directa de movimientos tácticos sin crear la lista legal completa cuando el rey no está en jaque, solo después de auditar equivalencia de reglas.
5. **TT y ordenamiento:** medir tasa real de reutilización y efectividad por heurística; no aumentar TT arbitrariamente en RP2040.
6. **Pico 2 W:** validar compilación ARM RP2350, Wi-Fi, LED y rendimiento con mismas posiciones; estudiar el beneficio de mayor SRAM sin cambiar comportamiento del motor.
7. **Pruebas funcionales:** suites perft ampliadas, FEN especiales, validación de repetición/enroque/en passant, regresión de puntuaciones Grumpy y legalidad de `bestmove`.
8. **Observabilidad:** consignar versión, `DEPTH`, `QDEPTH`, timeout y tamaño TT en el arranque; separar interrupción de cliente y vencimiento de tiempo en el campo `reason`.

Referencias técnicas

Primarias del proyecto: `src/board.cpp`, `src/engine.cpp`, `src/grumpy.cpp`, `src/pico_main.cpp`, `include/config.h`, `CMakeLists.txt`, `build.ps1`, `tools/*.py`, `reference/*.py`. Toda fórmula de arquitectura y comportamiento en este README describe esos fuentes. Las medidas empíricas proceden de pruebas comunicadas del hardware Pico W en octubre de 2026.

Fuentes oficiales generales: Raspberry Pi Pico SDK, documentación de Raspberry Pi microcontrollers, lwIP, protocolo UCI.

Nota sobre autoría y reproducción: Grumpy y Doc se documentan como componentes históricos integrados; su atribución original, licenciamiento y procedimiento de entrenamiento deben verificarse con las fuentes del proyecto de origen antes de una redistribución pública. El README no sustituye esa verificación.